

## Meeting Script: Autonomous Coding Agent – Requirements Gathering

Participants:

Product Manager (PM)

Client

PM: Good morning! Thank you for joining today's meeting. We're here to discuss your idea for an Autonomous Coding Agent. Shall we begin?

Client: Good morning! Yes, absolutely. I'm excited to share the vision and explore how we can bring it to life.

PM: What is the primary purpose of this product?

Client: The purpose of this Autonomous Coding Agent is to automate software development tasks such as generating code, writing tests, debugging issues, and creating pull requests. It aims to improve developer productivity and reduce development time.

PM: Could you provide a high-level overview of the proposed solution?

Client: Certainly. The solution is an AI-powered agent that understands user requirements, generates high-quality code, validates it through automated tests, and integrates seamlessly with GitHub. It will leverage advanced AI models and agentic frameworks to operate autonomously.

PM: Who are the intended users of this product?

Client: The primary users include software developers, startups, tech companies, and engineering teams looking to automate and accelerate their development workflows.

PM: Can you describe the target market and demographics?

Client: The product targets global users, particularly in America, Europe, and Asia. It caters to technology-driven organizations, **SaaS companies, freelance developers, and enterprises** seeking AI-powered development solutions.

PM: Could you share some example user personas?

Client: Of course.

Startup Founder: Needs rapid prototyping with limited resources.

Software Developer: Wants to automate repetitive coding tasks.

DevOps Engineer: Seeks seamless CI/CD and repository automation.

Engineering Manager: Aims to enhance team productivity and reduce costs.

PM: In what environment will users interact with the product?

Client: Users will primarily operate in cloud-based environments using modern development tools such as GitHub, Docker, and web-based dashboards. The solution will integrate into existing development ecosystems.

PM: What are the key needs of your users?

Client: Users require:

Automated code generation

Intelligent debugging and optimization

Automated test case creation

GitHub integration for version control

Secure and scalable solutions

Faster development cycles with minimal manual intervention

PM: Please provide a brief overview of the product.

Client: The Autonomous Coding Agent is an AI-powered development assistant that autonomously writes, tests, debugs, and deploys code. It acts as a virtual software engineer, enabling teams to build applications efficiently and accurately.

PM: How would you position this product in the market?

Client: For developers and organizations seeking faster and smarter software development, the Autonomous Coding Agent is an AI-powered assistant that automates coding and DevOps workflows. Unlike traditional development tools, it leverages agentic AI to deliver intelligent, end-to-end automation.

PM: Could you summarize the core capabilities of the product?

Client: Certainly. The key capabilities include:

- Requirement-to-code generation
- Automated testing and debugging
- GitHub repository integration
- Pull request creation and management
- Containerized execution using Docker
- Intelligent decision-making using LangGraph
- Continuous learning and optimization

PM: What pricing model do you envision?

Client: We are considering a flexible pricing strategy:

- Freemium Model: Limited features for individuals.
- Subscription Plans: Monthly and annual tiers for professionals and teams.
- Enterprise Licensing: Custom pricing with advanced features and support.
- Usage-Based Pricing: Charges based on API usage and compute resources.

PM: Could you outline the key product features?

Client: The core features include:

- AI-powered code generation
- Automated bug detection and fixing
- Intelligent test case generation
- GitHub integration for commits and pull requests
- Multi-language support
- Natural language requirement input
- Docker-based sandboxed execution
- CI/CD pipeline compatibility

Code review and optimization suggestions  
Interactive web dashboard

PM: What technology stack do you propose for this solution?

Client: The proposed tech stack includes:

AI Models: OpenAI  
Agent Framework: LangGraph  
Version Control Integration: GitHub API  
Containerization: Docker  
Backend: Python (FastAPI)  
Frontend: Next.js or React  
Database: PostgreSQL  
Deployment: AWS, Vercel, or Render  
Vector Database: FAISS or Pinecone

PM: Thank you for the detailed insights. This provides a strong foundation for defining the product requirements and architecture. Is there anything you'd like to add?

Client: No, this covers the core vision. I look forward to the next steps.

PM: Excellent. We will proceed with the requirements documentation and system design. Thank you for your time.

Client: Thank you

**Next meeting questions and random suggestions by clients, we start from here...**

**Some tips or idea can take but feel free to answer or choose**

## 15. System Architecture & Workflow Design

PM: Can we define how the system will technically operate end-to-end?

Client: Discuss agent flow (input → planning → code generation → testing → PR), microservices vs monolith, event-driven pipelines, and how LangGraph agents interact.

## 16. Security & Compliance

PM: What security measures and compliance requirements should we consider?

Client: Cover code sandboxing, data privacy, API security, OAuth with GitHub, secrets management, and compliance standards (e.g., GDPR, SOC2 if targeting enterprises).

## 17. Performance, Scalability & Limits

PM: How should the system perform under load and scale over time?

Client: Define concurrency (multiple agents), response time expectations, rate limits, horizontal scaling (Kubernetes/Docker), and handling large repositories.

## 18. Monitoring, Logging & Error Handling

PM: How will we monitor system performance and handle failures?

Client: Include logging systems, observability (metrics, traces), alerting, retry mechanisms, failure recovery, and audit trails for agent action